Conclusiones recomendaciones y anexos de Rapitrip

Complemento del entregable final de Diseño de Sistemas en Internet.

Conclusión

El desarrollo de este entregable permitió reunir en un mismo documento los requerimientos, modelos y decisiones técnicas de Rapitrip. Los casos de uso y diagramas describen las actividades de clientes, comercios, personal operativo y repartidores; a su vez, el modelo de datos y la arquitectura muestran cómo se relacionan esas actividades con la información y los componentes del sistema. Esta integración facilita seguir una necesidad del negocio desde su planteamiento hasta la solución diseñada para atenderla.

Durante el avance del proyecto se precisó la estructura inicialmente prevista. Rapitrip se organiza en tres aplicaciones web —tienda, consola y aplicación del repartidor— que comparten un backend administrado en Supabase. En ese backend se concentran los datos, los controles de acceso y las operaciones que deben mantener resultados consistentes, mientras que cada aplicación presenta las funciones correspondientes a sus usuarios. La división de la base de datos por responsabilidades, junto con las políticas de seguridad y las funciones destinadas a operaciones críticas, da una base clara para continuar el desarrollo.

El documento también deja visibles las dependencias de esta decisión: un cambio en los datos o en las funciones compartidas puede afectar a más de una aplicación. Por ello, la utilidad de los modelos no termina con su elaboración. Su valor depende de mantenerlos acordes con las migraciones, las interfaces y las pruebas conforme evolucione el proyecto. En este sentido, el entregable constituye una referencia para revisar lo construido y orientar las siguientes etapas, aunque la verificación del funcionamiento en los distintos entornos debe sustentarse con resultados de prueba y evidencia de operación.

Recomendaciones

1. Mantener la relación entre requerimientos, modelos y componentes del sistema. Se recomienda elaborar una matriz de trazabilidad en la que cada requerimiento tenga asociados sus casos de uso, diagramas, módulos y pruebas correspondientes. De esta manera, cuando se modifique un proceso, el equipo podrá identificar qué partes del diseño y del código deben revisarse. Asimismo, la matriz ayudará a detectar funciones descritas en el documento que todavía no cuenten con una verificación definida.

2. Actualizar la documentación al mismo ritmo que el sistema. Cada cambio relevante en las migraciones, las reglas de negocio o las aplicaciones debe acompañarse de una revisión de los diagramas y las descripciones técnicas afectadas. En particular, conviene presentar de forma uniforme la solución del repartidor como una aplicación web, ya que algunas referencias proceden de la propuesta inicial de una aplicación móvil futura. Esto permitirá que el documento siga representando la arquitectura utilizada.

3. Conservar evidencia verificable de las pruebas. Además de mantener los archivos de prueba en el repositorio, se recomienda registrar la fecha de ejecución, la versión del código, el entorno y el resultado obtenido. La revisión debe abarcar los permisos de cada actor y los recorridos principales de compra, pago, coordinación y entrega. Así, será posible distinguir entre una prueba diseñada y una función cuyo comportamiento ya fue comprobado en condiciones determinadas.

4. Validar los cambios en un entorno previo a producción. Antes de publicar nuevas funciones o migraciones, conviene comprobar que las tres aplicaciones siguen trabajando correctamente con el backend compartido. Esta validación debe incluir tanto los recorridos normales como situaciones de error, por ejemplo, pagos rechazados, pedidos cancelados, falta de repartidores disponibles o interrupciones en servicios externos. Con ello se reduce la posibilidad de que un cambio en un componente afecte procesos de otra aplicación sin advertirlo.

5. Dar seguimiento al funcionamiento real de la plataforma. Una vez que el sistema opere con usuarios y pedidos reales, se recomienda observar la disponibilidad, los tiempos de respuesta, los errores de integración y las incidencias de los procesos principales. Estos datos permitirán priorizar correcciones y valorar si la estructura actual necesita ajustes. Por tanto, cualquier decisión de separar componentes o ampliar la infraestructura podrá apoyarse en problemas observados y no solo en previsiones.

Anexo A Estructura y versión del proyecto

La estructura actual reúne la tienda web, la consola y la aplicación del repartidor en un mismo repositorio. Cada aplicación conserva sus pantallas y responsabilidades, mientras que los paquetes compartidos agrupan componentes de interfaz, validaciones, tipos y acceso a servicios comunes. La imagen siguiente resume cómo esas aplicaciones utilizan el backend administrado en Supabase.

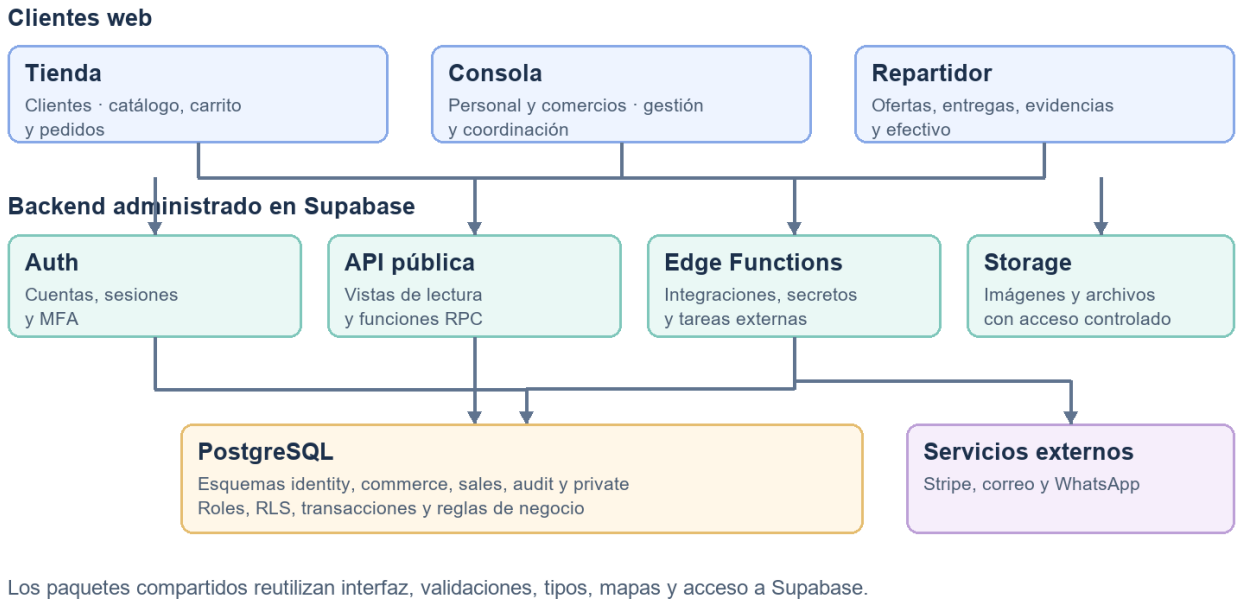


Figura A1. Aplicaciones web y componentes del backend de Rapitrip.

Estructura de referencia: apps/store, apps/console y apps/dealer para las aplicaciones; packages para el código compartido; y supabase para migraciones, funciones y pruebas de base de datos. Esta vista corresponde al material disponible en el proyecto al preparar el anexo.

Anexo B Modelo de datos y migraciones

El modelo entidad–relación muestra los vínculos entre las áreas de identidad, comercios, catálogo, inventario, pedidos, entregas, pagos y auditoría. Su fuente de contraste son las migraciones SQL conservadas en supabase/migrations. Como el esquema puede evolucionar, el diagrama debe revisarse junto con esas migraciones cuando se agreguen o modifiquen tablas y relaciones.

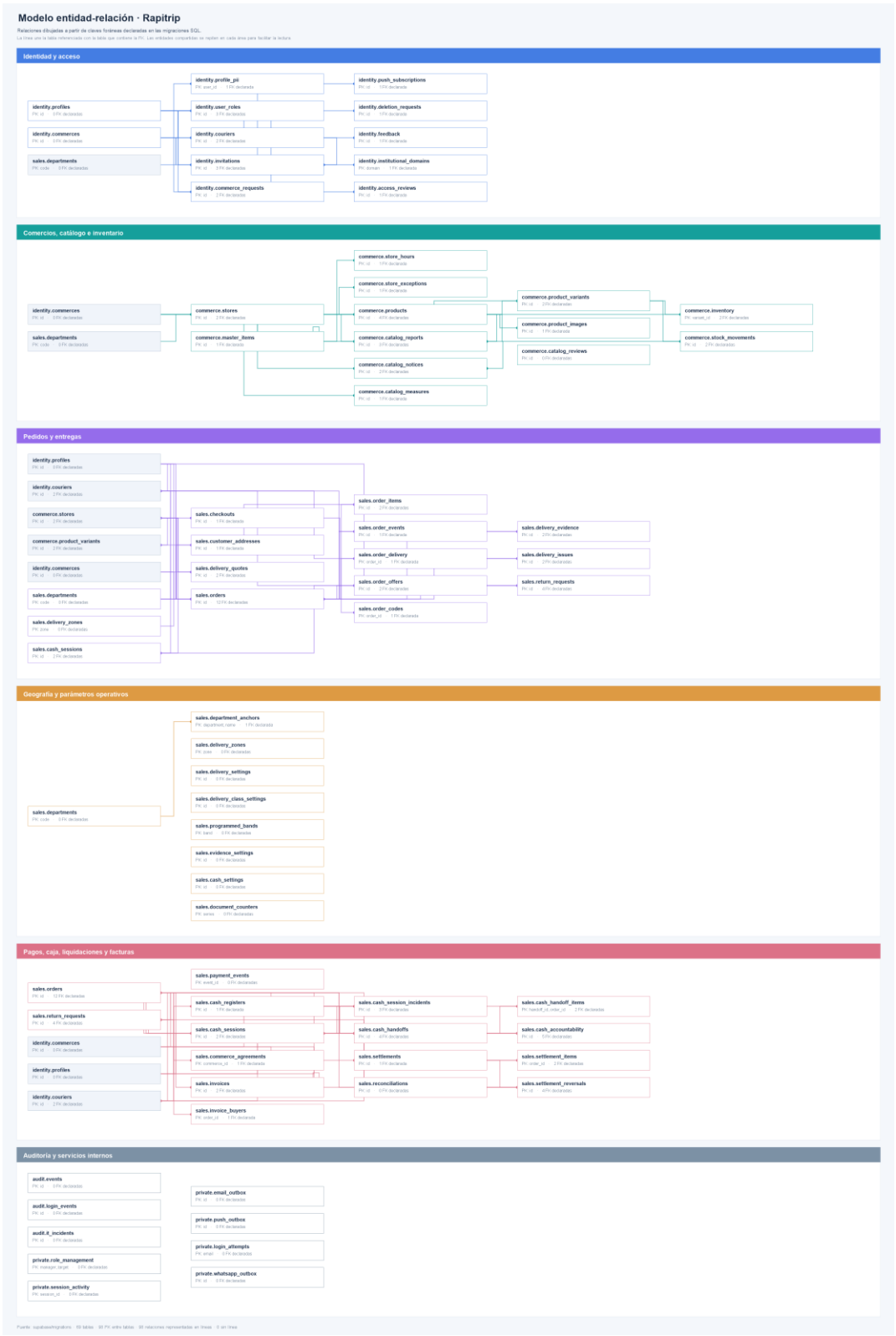


Figura B1. Modelo entidad–relación de Rapitrip con relaciones representadas por líneas.

Anexo C Repositorio del proyecto

https://github.com/richard0956/Proyecto-Rapitrip.git

Anexo D Prototipos y recorridos principales

El entregable original remite a los prototipos de Figma para consultar los recorridos del cliente, la consola de operación y el repartidor. Los siguientes enlaces permiten revisar la navegación y las pantallas de cada actor.

Vista general de los prototipos

Prototipo de la tienda web

Prototipo de la consola

Prototipo del repartidor